

Vel Tech Group of Institutions

Name: Gontu Yaswanth

Email: vtu33434@veltech.edu.in

Roll no: Vtu33434

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Competitive Coding II_L8

VTU_Week 3_CY

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Dhiya is a data analyst working for a data compression company. One of her tasks is to analyze logs that are compressed using a simple format where each character is followed by the number of times it appears. For example, "A3B2" represents the string "AAABB".

Her goal is to identify whether a specific pattern exists in the original, uncompressed message and, if so, determine all starting and ending indices of the pattern's occurrence in the expanded string. To perform this efficiently, Dhiya decides to first expand the compressed string and then apply pattern matching.

Your task is to help Dhiya by building a program using Boyer-Moore pattern matching algorithm that performs this operation.

Input Format

The first line of input contains a string *s* consisting of characters followed by digits representing the compressed text.

The second line of input contains a string *p* representing the pattern string.

Output Format

The first line of output prints "Expanded Text: *X*" where *X* represents the expanded text.

The second line of output prints "Matches found (start_index end_index):" if match found.

For each match of the pattern in the expanded text, print the starting and ending indices (0-based) separated by a space (Print one pair per line).

If no match is found, print "No match found."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: A3B2C4D1

CCC

Output: Expanded Text: AAABBCCCCD

Matches found (start_index end_index):

5 7

6 8

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <ctype.h>
```

```
#define MAX 256
```

```
// Expand compressed string
```

```
void expand(char *s, char *res) {
```

```

int i = 0, k = 0;
while (s[i]) {
    char ch = s[i++];
    int num = 0;
    while (isdigit(s[i])) {
        num = num * 10 + (s[i] - '0');
        i++;
    }
    for (int j = 0; j < num; j++)
        res[k++] = ch;
}
res[k] = '\0';
}

```

```

// Bad character table
void badChar(char *pat, int m, int bad[]) {
    for (int i = 0; i < MAX; i++)
        bad[i] = -1;
    for (int i = 0; i < m; i++)
        bad[(int)pat[i]] = i;
}

```

```

// Boyer-Moore search
void search(char *txt, char *pat) {
    int n = strlen(txt), m = strlen(pat);
    int bad[MAX];
    badChar(pat, m, bad);

    int found = 0;
    int s = 0;

    while (s <= n - m) {
        int j = m - 1;

        while (j >= 0 && pat[j] == txt[s + j])
            j--;

        if (j < 0) {
            printf("%d %d\n", s, s + m - 1);
            found = 1;
            s += 1; // allow overlapping
        } else {

```

```

        int shift = j - bad[tst[s + j]];
        s += (shift > 1) ? shift : 1;
    }
}

if (!found)
    printf("No match found.");
}

int main() {
    char s[1001], p[1001], txt[5000];

    scanf("%s", s);
    scanf("%s", p);

    expand(s, txt);

    printf("Expanded Text: %s\n", txt);

    int n = strlen(txt), m = strlen(p);

    int exists = 0;
    for (int i = 0; i <= n - m; i++) {
        if (strncmp(&txt[i], p, m) == 0) {
            exists = 1;
            break;
        }
    }

    if (exists) {
        printf("Matches found (start_index end_index):\n");
        search(txt, p);
    } else {
        printf("No match found.");
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Sanya is developing a text analysis tool and wants to identify the longest palindromic sequence in a given string. A palindrome is a string that reads the same forwards and backwards.

Your task is to write a program that takes a string as input and returns the longest palindromic substring it contains. If there are multiple substrings of the same maximum length, returning any one of them is acceptable.

Input Format

The input consists of a single line containing the string S.

S may consist of uppercase and lowercase English letters.

Output Format

The output prints a single string representing the longest palindromic substring found in S.

Refer to the sample input and output for format specifications.

Sample Test Case

Input: babad

Output: bab

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>
```

```
int main() {
    char s[101];
    scanf("%s", s);

    int n = strlen(s);
    int start = 0, maxLen = 1;
```

```

for (int i = 0; i < n; i++) {
    // Odd length
    int l = i, r = i;
    while (l >= 0 && r < n && s[l] == s[r]) {
        if (r - l + 1 > maxLen) {
            start = l;
            maxLen = r - l + 1;
        }
        l--; r++;
    }

    // Even length
    l = i; r = i + 1;
    while (l >= 0 && r < n && s[l] == s[r]) {
        if (r - l + 1 > maxLen) {
            start = l;
            maxLen = r - l + 1;
        }
        l--; r++;
    }
}

for (int i = start; i < start + maxLen; i++)
    printf("%c", s[i]);

return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Rita is working on string pattern matching using the Z-Algorithm. She wants to find all starting indices where a given pattern string occurs in a text string efficiently.

Write a program that takes a text string and a pattern string as input and outputs a list of indices (0-based) where the pattern occurs. If the pattern does not occur in the text, output an empty list.

Input Format

The first line contains the text string txt.

The second line contains the pattern string pat.

Output Format

The output prints a list of integers representing the starting indices of all occurrences of the pattern in the text.

If no match is found, print [].

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: abesdu
edu

Output: []

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>
```

```
int main() {
    char txt[1001], pat[101], s[1200];
    int Z[1200] = {0};
```

```
    scanf("%s", txt);
    scanf("%s", pat);
```

```
    // Combine string
    sprintf(s, "%s%s", pat, txt);
```

```
    int n = strlen(s);
    int L = 0, R = 0;
```

```
    for (int i = 1; i < n; i++) {
        if (i <= R)
```

```

    Z[i] = (R - i + 1 < Z[i - L]) ? (R - i + 1) : Z[i - L];

    while (i + Z[i] < n && s[Z[i]] == s[i + Z[i]])
        Z[i]++;

    if (i + Z[i] - 1 > R) {
        L = i;
        R = i + Z[i] - 1;
    }
}

int m = strlen(pat);
int found = 0;

printf("[");

for (int i = 0; i < n; i++) {
    if (Z[i] == m) {
        if (found) printf(", ");
        printf("%d", i - m - 1);
        found = 1;
    }
}

printf("]");

return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Neha is building a plagiarism-detection feature where she must compare two text strings to find similarities. To efficiently detect overlaps between the strings, she uses the KMP (Knuth–Morris–Pratt) string matching algorithm, relying on the LPS (Longest Prefix Suffix) array as the main data structure.

Given two strings, write a program to ensure that the task is done by

finding the longest common substring present in both strings. If no common substring exists, the program should clearly indicate that result.

Input Format

The first line contains a string representing the first text.

The second line contains a string representing the second text.

Output Format

The output prints the longest common substring if it exists.

If there is no common substring, print "There is no common substring".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: abcdef

abc

Output: abc

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {  
    char s1[101], s2[101];  
    scanf("%s", s1);  
    scanf("%s", s2);
```

```
    int n = strlen(s1), m = strlen(s2);
```

```
    int maxLen = 0, start = -1;
```

```
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < m; j++) {  
            int k = 0;
```

```

while (i + k < n && j + k < m && s1[i + k] == s2[j + k]) {
    k++;
    if (k > maxLen) {
        maxLen = k;
        start = i;
    }
}
}
}

if (maxLen == 0) {
    printf("There is no common substring");
} else {
    for (int i = start; i < start + maxLen; i++)
        printf("%c", s1[i]);
}

return 0;
}

```

Status : Correct

Marks : 10/10